

Rainfall Prediction – Weather Forecasting for Agriculture

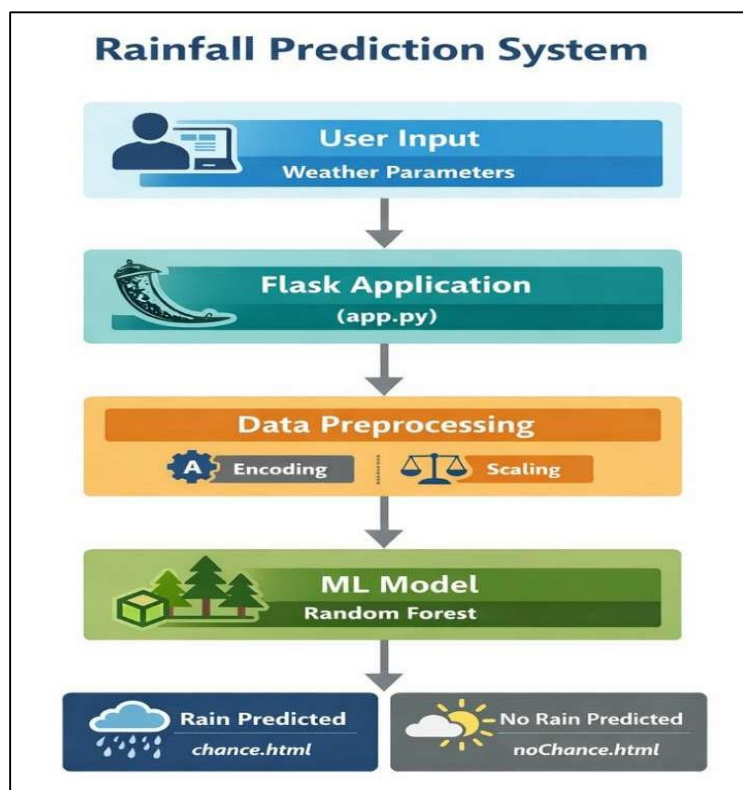
Project Description:

Rainfall plays a crucial role in agriculture, water resource management, and climate planning. Predicting rainfall accurately helps farmers, agricultural experts, and policymakers make informed decisions regarding crop planning, irrigation scheduling, and disaster management.

Rainfall prediction is a complex task due to the dynamic nature of weather conditions. By leveraging Machine Learning techniques, we can analyze historical weather data and predict whether it will rain tomorrow.

In this project, we use classification algorithms such as Decision Tree, Random Forest, Support Vector Machine (SVM), Gradient Boosting, and XGBoost. The best-performing model is selected based on accuracy and evaluation metrics and saved in .pkl format. The model is integrated into a Flask web application and deployed for real-time prediction.

Technical Architecture:



Pre requisites:

To complete this project, you must required following software's, concepts and packages

- **Anaconda navigator and pycharm:**
 - Refer the link below to download anaconda navigator
 - Link : <https://youtu.be/1ra4zH2G4o0>
- **Python packages:**
 - Open anaconda prompt as administrator
 - Type “pip install numpy” and click enter.
 - Type “pip install pandas” and click enter.
 - Type “pip install scikit-learn” and click enter.
 - Type ”pip install matplotlib” and click enter.
 - Type ”pip install scipy” and click enter.
 - Type ”pip install pickle-mixin” and click enter.
 - Type ”pip install seaborn” and click enter.
 - Type “pip install Flask” and click enter.
 - Type “pip install xgboost” and click enter.

Prior Knowledge:

You must have prior knowledge of following topics to complete this project.

- **ML Concepts**
 - Supervised learning: <https://www.javatpoint.com/supervised-machine-learning>
 - Unsupervised learning: <https://www.javatpoint.com/unsupervised-machine-learning>
 - Regression and classification
 - Decision tree: <https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm>
 - Random forest: <https://www.javatpoint.com/machine-learning-random-forest-algorithm>
 - KNN: <https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning>
 - Xgboost: <https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/>
 - Evaluation metrics: <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
- **Flask Basics** : https://www.youtube.com/watch?v=Ij4I_CvBnt0

Project Objectives:

By the end of this project you will:

- Understand data preprocessing techniques
- Perform exploratory data analysis
- Apply classification algorithms
- Compare multiple ML models

- Deploy ML model using Flask
- Understand real-world ML deployment workflow

Project Flow:

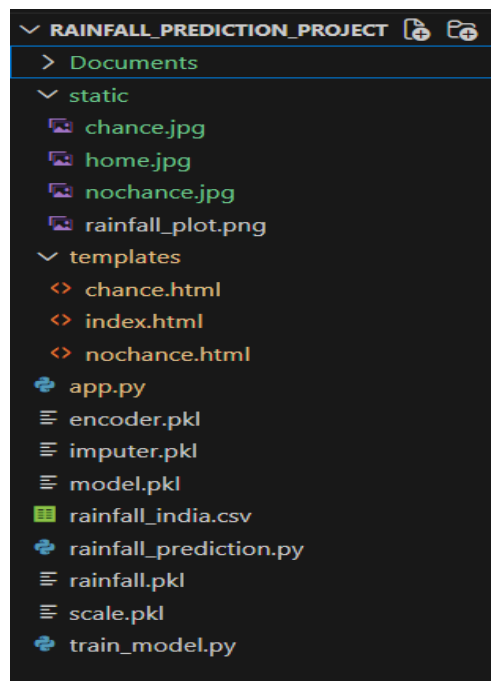
- User interacts with the UI and enters weather parameters.
- The input values are sent to the Flask backend.
- The backend preprocesses the input (encoding + scaling).
- The trained model predicts rainfall.
- Prediction result is displayed on UI.

To accomplish this, we have to complete all the activities listed below,

- Data collection
 - Collect the dataset or create the dataset
- Visualizing and analyzing data
 - Univariate analysis
 - Bivariate analysis
 - Multivariate analysis
 - Descriptive analysis
- Data pre-processing
 - Checking for null values
 - Handling outlier
 - Handling categorical data
 - Splitting data into train and test
- Model building
 - Import the model building libraries
 - Initializing the model
 - Training and testing the model
 - Evaluating performance of model
 - Save the model
- Application Building
 - Create an HTML file
 - Build python code

Project Structure:

Create the Project folder which contains files as shown below



- We are building a flask application which needs HTML pages stored in the templates folder and a python script app.py for scripting.
- rainfall.pkl is our saved model. Further we will use this model for flask integration.
- Training folder contains model training files and templates folder contains IBM deployment files.

Milestone 1: Data Collection

ML depends heavily on data, It is most crucial aspect that makes algorithm training possible. So this section allows you to download the required dataset.

Activity 1: Download the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

In this project we have used WeatherAUS.csv data. This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Link:

https://docs.google.com/spreadsheets/d/1RA2OO0LZTeQyKI_mvnsAjp6LM4YzWI1Tz0SUG5-Ao/edit?gid=121883362#gid=121883362

The dataset contains historical weather observations including:

- Location
- Temperature
- Rainfall
- Humidity
- Pressure
- Wind Speed
- Rain Tomorrow (Target Variable)

Milestone 2: Data Visualizing and analysis

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analysing techniques.

Note: There is n number of techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Activity 1: Importing the libraries

Import the necessary libraries as shown in the image. (optional) Here we have used visualization style as fivethirtyeight.

```
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV
import imblearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
```

Activity 2: Read the Dataset

Our dataset format might be in .csv, excel files, .txt, .json, etc. We can read the dataset with the help of pandas.

In pandas we have a function called `read_csv()` to read the dataset. As a parameter we have to give the directory of csv file.

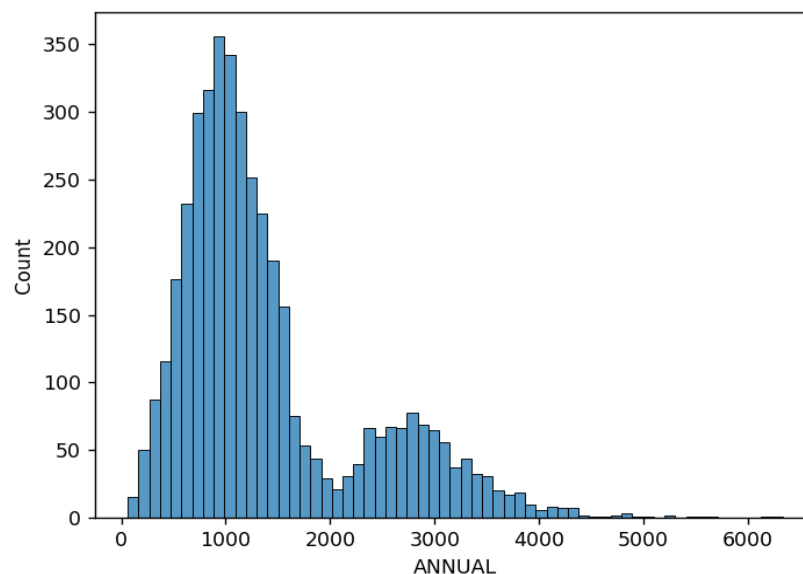
	YEAR	JAN	FEB	MAR	APR	...	ANNUAL	JF	MAM	JJAS	OND
count	4188.000000	4184.000000	4185.000000	4182.000000	4184.000000	...	4162.000000	4182.000000	4179.000000	4178.000000	4175.000000
mean	1959.220630	18.938337	21.599427	27.412912	43.071845	...	1409.449207	40.522979	155.820914	1063.851580	153.648623
std	33.717097	33.793128	35.731734	46.927963	68.150985	...	902.599421	59.293904	201.198627	706.666045	166.863171
min	1901.000000	0.000000	0.000000	0.000000	0.000000	...	62.300000	0.000000	0.000000	57.400000	0.000000
25%	1930.000000	0.600000	0.500000	1.000000	3.000000	...	803.025000	4.000000	24.000000	573.850000	34.000000
50%	1959.000000	5.950000	6.500000	7.900000	15.450000	...	1120.350000	18.900000	74.800000	880.600000	97.700000
75%	1988.000000	22.000000	26.600000	31.375000	49.650000	...	1643.575000	50.175000	196.950000	1287.450000	211.800000
max	2017.000000	583.700000	403.500000	605.600000	595.100000	...	6331.100000	699.500000	1745.800000	4536.900000	1252.500000

[8 rows x 18 columns]

Activity 3: Univariate analysis

In simple words, univariate analysis is understanding the data with single feature. Here we have displayed two different graphs such as distribution plot.

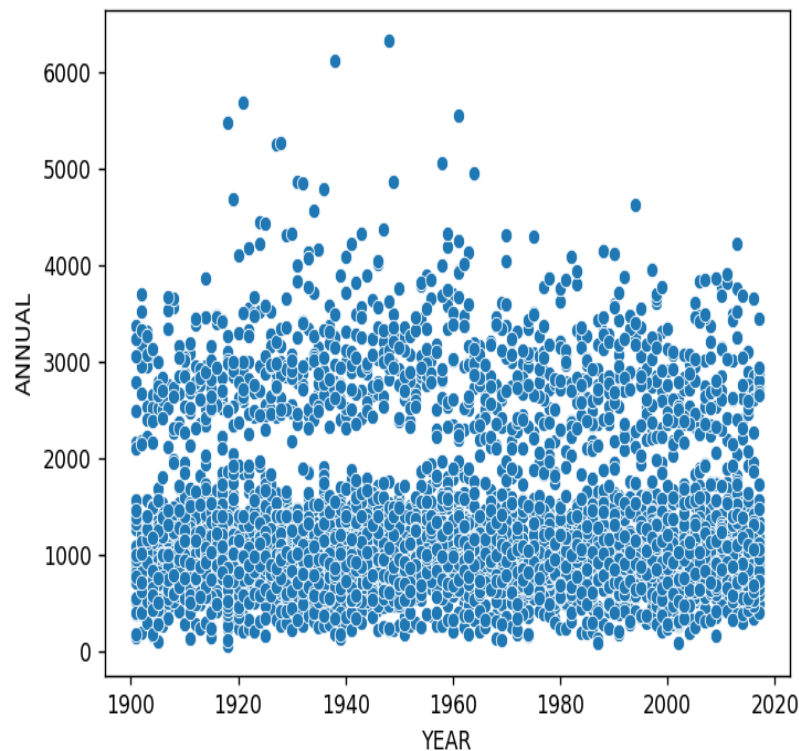
- Seaborn package provides a wonderful function `boxplot`. With the help of `count plot`, we can find the distribution of the feature. To make multiple graphs in a single plot, we use `subplot`.



- In our dataset we have some categorical features. With the count plot function, we are going to count the unique category in those features. We have created a dummy data frame with categorical features. With for loop and subplot we have plotted this below graph.
- From the plot we came to know, Applicants income is skewed towards left side, where as credit history is categorical with 1.0 and 0.0
-

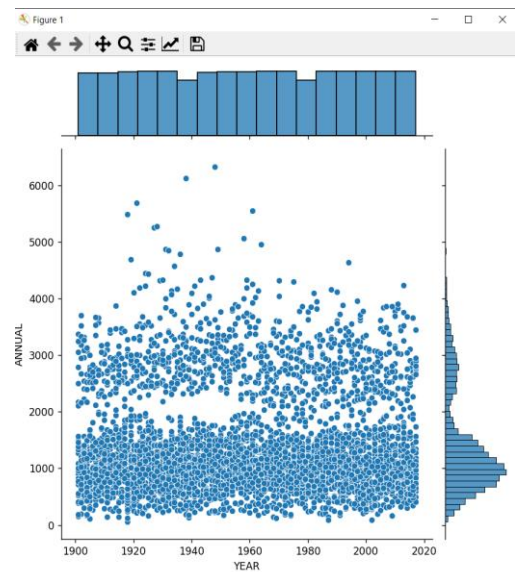
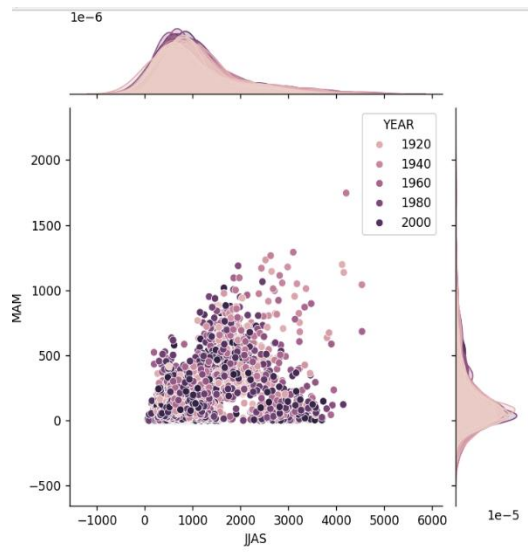
Activity 4: Bivariate analysis

Scatter plot:-



Joint Plot:

A joint plot can be thought of as a stogram across a categorical, instead of quantitative, variable. The basic API and options are identical to those for `barplot()`, so you can compare counts across nested variables.

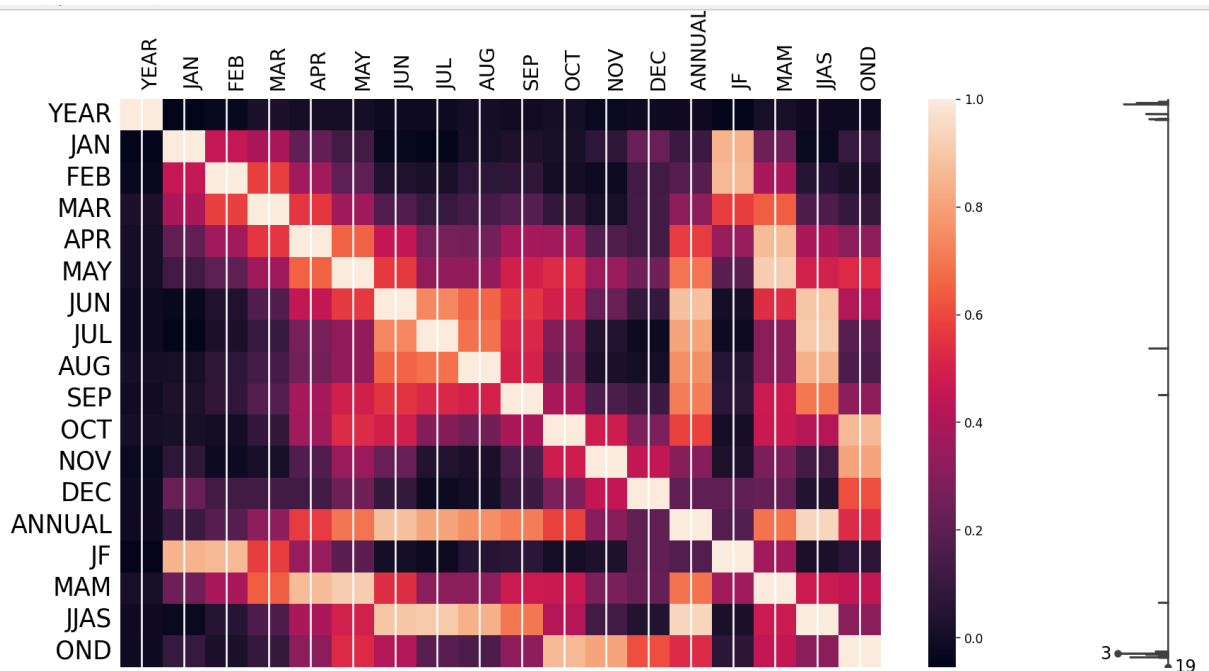


From the above graph we can infer the analysis such as

- Segmenting the whether column and temperature column based on bar graphs
- Segmenting the Rainy day and No rain tomorrow based on bar graphs ,for drawing insights such as Rainy or not.

Activity 5: Multivariate analysis

In simple words, multivariate analysis is to find the relation between multiple features. Here we have used heatmap from seaborn package.



Activity 6: Descriptive analysis

Descriptive analysis is to study the basic features of data with the statistical process. Here pandas has a worthy function called describe. With this describe function we can understand the unique, top and frequent values of categorical features. And we can find mean, std, min, max and percentile values of continuous features.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4188 entries, 0 to 4187
Data columns (total 19 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   SUBDIVISION 4188 non-null   object
1   YEAR        4188 non-null   int64
2   JAN         4184 non-null   float64
3   FEB         4185 non-null   float64
4   MAR         4182 non-null   float64
5   APR         4184 non-null   float64
6   MAY         4185 non-null   float64
7   JUN         4183 non-null   float64
8   JUL         4181 non-null   float64
9   AUG         4184 non-null   float64
10  SEP         4182 non-null   float64
11  OCT         4181 non-null   float64
12  NOV         4177 non-null   float64
13  DEC         4178 non-null   float64
14  ANNUAL      4162 non-null   float64
15  JF          4182 non-null   float64
16  MAM         4179 non-null   float64
17  JJAS        4178 non-null   float64
18  OND         4175 non-null   float64
dtypes: float64(17), int64(1), object(1)
memory usage: 621.8+ KB
```

Milestone 3: Data Pre-processing

As we have understood how the data is lets pre-process the collected data.

The download data set is not suitable for training the machine learning model as it might have so much of randomness so we need to clean the dataset properly in order to fetch good results. This activity includes the following steps.

- Handling missing values
- Encoding categorical variables
- Feature scaling
- Splitting dataset into training and test set

Note: These are the general steps of pre-processing the data before using it for machine learning. Depending on the condition of your dataset, you may or may not have to go through all these steps.

Activity 1: Checking for null values

- Let's find the shape of our dataset first, To find the shape of our data, `df.shape` method is used. To find the data type, `df.info()` function is used.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4188 entries, 0 to 4187
Data columns (total 19 columns):
#   Column      Non-Null Count  Dtype
---  -
0   SUBDIVISION 4188 non-null   object
1   YEAR        4188 non-null   int64
2   JAN         4184 non-null   float64
3   FEB         4185 non-null   float64
4   MAR         4182 non-null   float64
5   APR         4184 non-null   float64
6   MAY         4185 non-null   float64
7   JUN         4183 non-null   float64
8   JUL         4181 non-null   float64
9   AUG         4184 non-null   float64
10  SEP         4182 non-null   float64
11  OCT         4181 non-null   float64
12  NOV         4177 non-null   float64
13  DEC         4178 non-null   float64
14  ANNUAL      4162 non-null   float64
15  JF          4182 non-null   float64
16  MAM         4179 non-null   float64
17  JJAS        4178 non-null   float64
18  OND         4175 non-null   float64
dtypes: float64(17), int64(1), object(1)
memory usage: 621.8+ KB
```

- For checking the null values, `df.isnull()` function is used. To sum those null values we use `.sum()` function to it. From the below image we found that there are no null values present in our dataset. So we can skip handling of missing values step.

Activity 2: Handling Categorical Values

As we can see our dataset has categorical data we must convert the categorical data to integer encoding or binary encoding.

To convert the categorical features into numerical features we use encoding techniques. There are several techniques but in our project we are using manual encoding with the help of list comprehension.

- In our project, so many data Location With list comprehension encoding is done.

```
from sklearn.preprocessing import LabelEncoder

encoders = {}

for col in cat_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    encoders[col] = le
```

Activity 3: Balancing the dataset:

Data Balancing is one of the most important step, which need to be performed for classification models, because when we train our model on imbalanced dataset , we will get biased results, which means our model is able to predict only one class element

```
selected_features = [
    'Location',
    'MinTemp',
    'MaxTemp',
    'Rainfall',
    'Humidity9am',
    'Humidity3pm',
    'Pressure9am',
    'Pressure3pm'
]

X = data[selected_features]
y = data['RainTomorrow']

print(X.columns)

Index(['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Humidity9am',
       'Humidity3pm', 'Pressure9am', 'Pressure3pm'],
      dtype='object')

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

From the above picture, we can infer that ,previously our dataset is having many classes now we get some class items, after applying smote technique on the dataset the size has been changed for temperature class.

Activity 4: Scaling the Data

Scaling is one the important process, we have to perform on the dataset, because of data measures in different ranges can leads to mislead in prediction

Models such as KNN, Logistic regression need scaled data, as they follow distance based method and Gradient Descent concept.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

We will perform scaling only on the input values

Once the dataset is scaled, it will be converted into array and we need to convert it back to dataframe.

Activity : Splitting data into train and test

Now let's split the Dataset into train and test sets

Changes: first split the dataset into x and y and then split the data set

Here x and y variables are created. On x variable, df is passed with dropping the target variable. And on y target variable is passed. For splitting training and testing data we are using train_test_split() function from sklearn. As parameters, we are passing x, y, test_size, random_state.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
```

Milestone 4: Model Building

Now our data is cleaned and it's time to build the model. We can train our data on different algorithms. for this project we are applying four classification algorithms. The best model is saved based on its performance.

Activity 1: Decision tree model

A function named decisionTree is created and train and test data are passed as the parameters. Inside the function, DecisionTreeClassifier algorithm is initialized and training data is passed to the model with .fit() function. Test data is predicted with .predict() function and saved in new variable. For evaluating the model, confusion matrix and classification report is done.

```
log_model = LogisticRegression(max_iter=1000)
dt_model = DecisionTreeClassifier()
rf_model = RandomForestClassifier()
svm_model = SVC()
gb_model = GradientBoostingClassifier()
xgb_model = XGBClassifier(use_label_encoder=True, eval_metric='logloss')
```

Activity 2: Random forest model

A function named randomForest is created and train and test data are passed as the parameters. Inside the function, RandomForestClassifier algorithm is initialized and training data is passed to the model with .fit() function. Test data is predicted with .predict() function and saved in new variable. For evaluating the model, confusion matrix and classification report is done.

```
log_model = LogisticRegression(max_iter=1000)
dt_model = DecisionTreeClassifier()
rf_model = RandomForestClassifier()
svm_model = SVC()
gb_model = GradientBoostingClassifier()
xgb_model = XGBClassifier(use_label_encoder=True, eval_metric='logloss')
```

Activity 3: Xgboost model

A function named xgboost is created and train and test data are passed as the parameters. Inside the function, GradientBoostingClassifier algorithm is initialized and training data is passed to the model with .fit() function. Test data is predicted with .predict() function and

saved in new variable. For evaluating the model, confusion matrix and classification report is done.

```
log_model = LogisticRegression(max_iter=1000)
dt_model = DecisionTreeClassifier()
rf_model = RandomForestClassifier()
svm_model = SVC()
gb_model = GradientBoostingClassifier()
xgb_model = XGBClassifier(use_label_encoder=True, eval_metric='logloss')
```

Now let's see the performance of all the models and save the best model

Activity 4: Compare the model

For comparing the above models compare Model function is defined.

```
print("Logistic Regression Accuracy:", accuracy_score(y_test, log_pred))
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
print("Gradient Boosting Accuracy:", accuracy_score(y_test, gb_pred))
print("XGBoost Accuracy:", accuracy_score(y_test, xgb_pred))
```

```
... Logistic Regression Accuracy: 0.8322219166781246
Decision Tree Accuracy: 0.7764333837481094
Random Forest Accuracy: 0.8455932902516156
SVM Accuracy: 0.8407122232916265
Gradient Boosting Accuracy: 0.842946514505706
XGBoost Accuracy: 0.8494431458820294
```

After calling the function, the results of models are displayed as output. From the models Xgboost is performing well. From the below image, We can see the accuracy of the model. Xgboost is giving the accuracy of 84.9% with training data , 81.1% accuracy for the testing data.so we considering xgboost and deploying this model.

Activity 5: Evaluating performance of the model and saving the model

From sklearn, `cross_val_score` is used to evaluate the score of the model. On the parameters, we have given `rf` (model name), `x`, `y`, `cv` (as 5 folds). Our model is performing well. So, we are saving the model by `pickle.dump()`.

Note: To understand cross validation, refer this link. <https://towardsdatascience.com/cross-validation-explained-evaluating-estimator-performance-e51e5430ff85>.

```
import pickle

pickle.dump(best_model, open("rainfall.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))
cat_cols = ['Location']

encoders = {}

for col in cat_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    encoders[col] = le

pickle.dump(encoders, open("encoder.pkl", "wb"))
pickle.dump(num_imputer, open("imputer.pkl", "wb"))
```

Milestone 5: Application Building

In this section, we will be building a web application that is integrated to the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI.

This section has the following tasks

- Building HTML Pages
- Building app.py

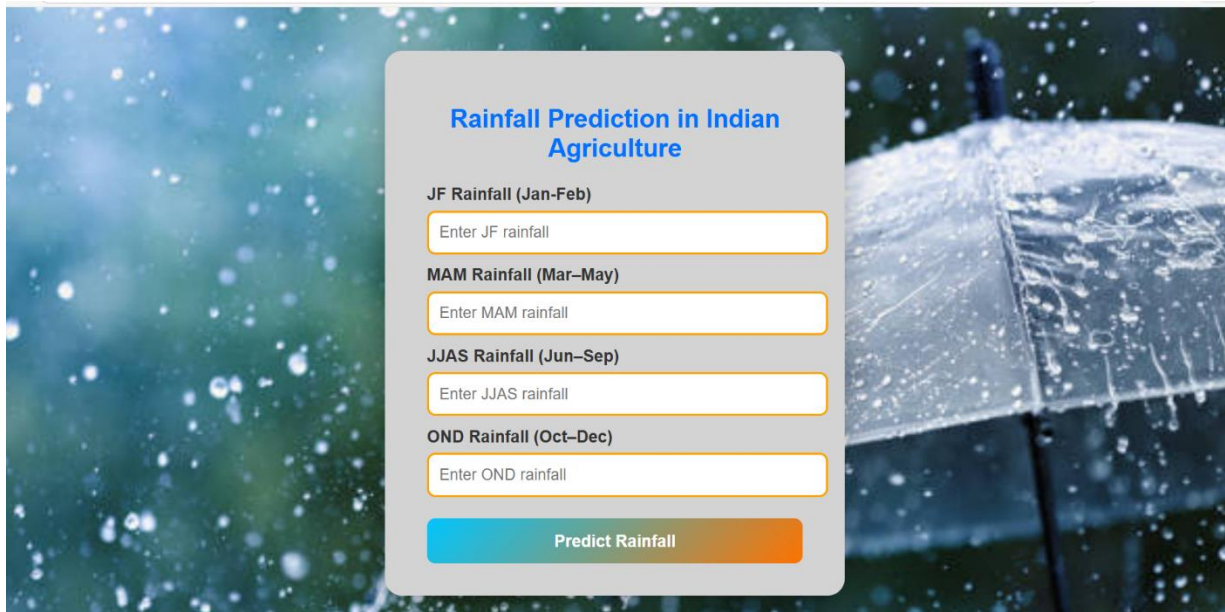
Activity1: Building Html Pages:

For this project create three HTML files namely

- Index.html
- Chance.html
- nochance.html

and save them in templates folder

Let's see how our Index.html page looks like this after input temperature humidity values and rain values then press the Predict Button to check the Result :

A screenshot of a web form titled "Rainfall Prediction in Indian Agriculture". The form is centered on a background image of a rainy scene with a transparent umbrella. It contains four input fields for different rainfall periods: "JF Rainfall (Jan-Feb)", "MAM Rainfall (Mar-May)", "JJAS Rainfall (Jun-Sep)", and "OND Rainfall (Oct-Dec)". Each field has a placeholder text "Enter [period] rainfall". At the bottom of the form is a blue button labeled "Predict Rainfall".

Rainfall Prediction in Indian Agriculture

JF Rainfall (Jan-Feb)

MAM Rainfall (Mar-May)

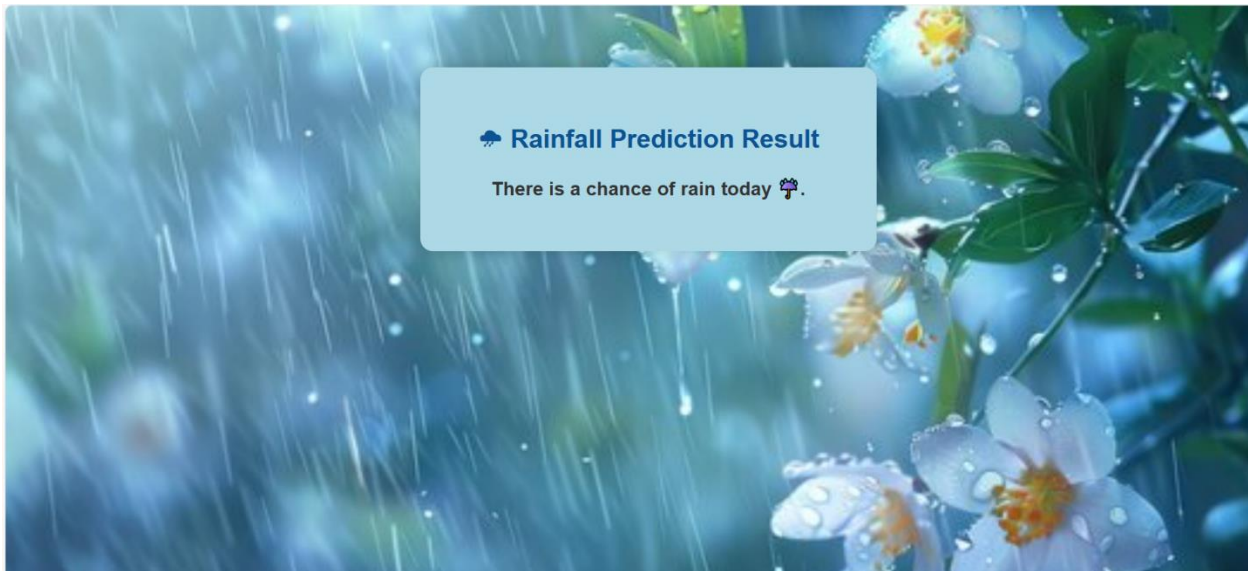
JJAS Rainfall (Jun-Sep)

OND Rainfall (Oct-Dec)

Predict Rainfall

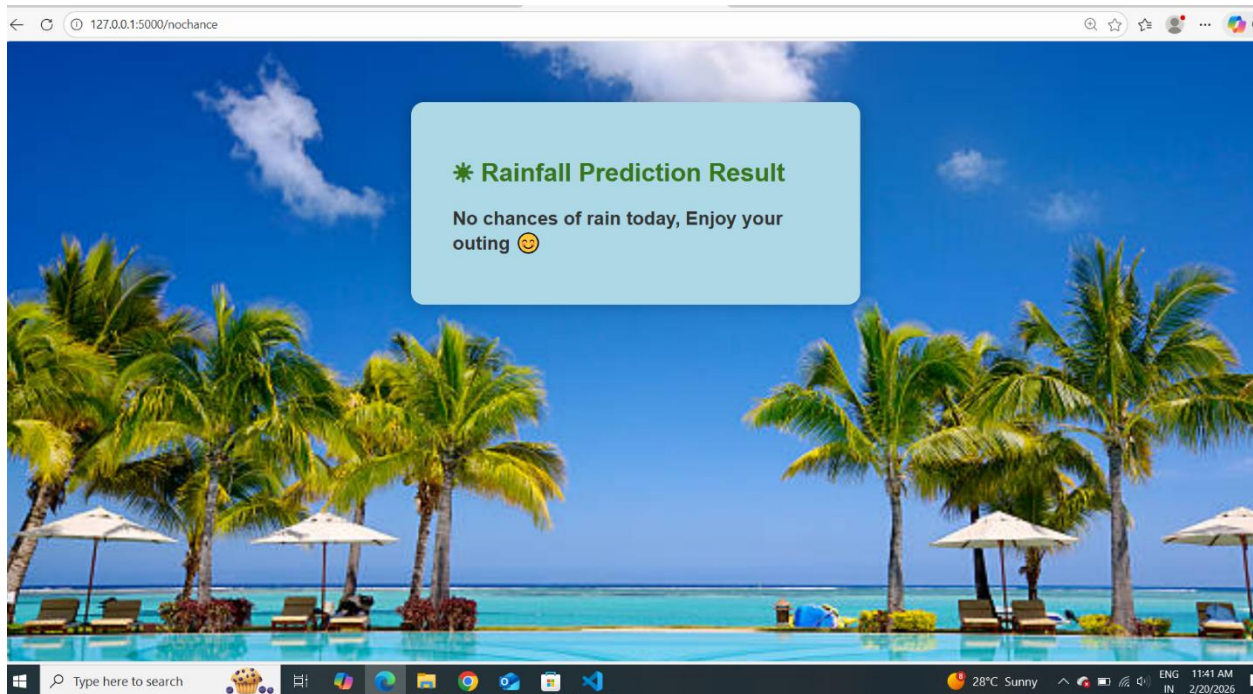
Now when you click on predict button bottom you will get redirected to html file result,

Lets look how our chance.html file looks like:



or

Lets look how our no chance.html file looks like:



Activity 2: Build Python code:

Import the libraries

```
from flask import Flask, render_template, request
import numpy as np
import pickle
import pandas as pd
```

Load the saved model. Importing flask module in the project is mandatory. An object of Flask class is our WSGI application. Flask constructor takes the name of the current module (`__name__`) as argument.

```
app = Flask(__name__)

# Load saved files
model = pickle.load(open(r"C:\Users\DELL\OneDrive\Desktop\Rainfall_Prediction\Saved_Models\rainfall.p", 'rb'))
scaler = pickle.load(open(r"C:\Users\DELL\OneDrive\Desktop\Rainfall_Prediction\Saved_Models\scaler.pkl", 'rb'))
encoders = pickle.load(open(r"C:\Users\DELL\OneDrive\Desktop\Rainfall_Prediction\Saved_Models\encoders.pkl", 'rb'))
```

Render HTML page:

Here we will be using declared constructor to route to the HTML page which we have created earlier.

```
@app.route('/')
def home():
    return render_template("index.html")
```

In the above example, '/' URL is bound with home.html function. Hence, when the home page of the web server is opened in browser, the html page will be rendered. Whenever you enter the values from the html page the values can be retrieved using POST Method.

Retrieves the value from UI:

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        input_data = request.form.to_dict()
        # Convert to DataFrame
        df = pd.DataFrame([input_data])

        # * Convert numeric columns to float
        numeric_cols = [
            'MinTemp',
            'MaxTemp',
            'Rainfall',
            'Humidity9am',
            'Humidity3pm',
            'Pressure9am',
            'Pressure3pm'
        ]

        for col in numeric_cols:
            if col in df.columns:
                df[col] = df[col].astype(float)

        for col in encoders:
            if col in df.columns:
                try:
                    df[col] = encoders[col].transform(df[col])
                except Exception as e:
                    return f"Encoding Error for {col}: {e}"

        scaled_data = scaler.transform(df)
        prediction = model.predict(scaled_data)[0]
        if prediction == 1:
            return render_template("chance.html")
        else:
            return render_template("noChance.html")
```

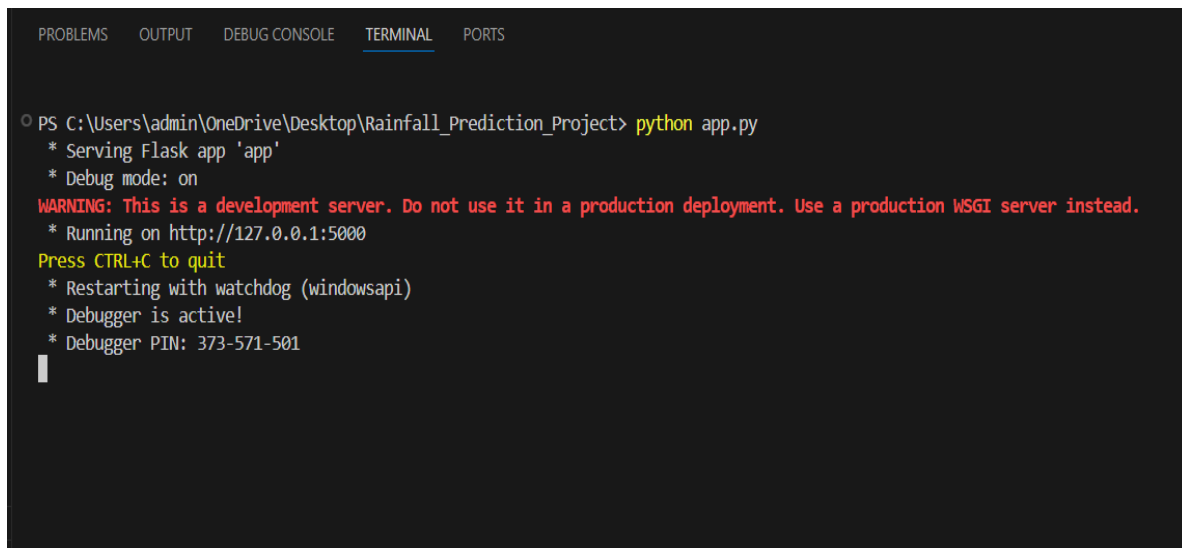
Here we are routing our app to predict() function. This function retrieves all the values from the HTML page using Post request. That is stored in an array. This array is passed to the model.predict() function. This function returns the prediction. And this prediction value will be rendered to the text that we have mentioned in the submit.html page earlier.

Main Function:

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Activity 3: Run the application

- Open anaconda prompt from the start menu
- Navigate to the folder where your python script is.
- Now type “python app.py” command
- Navigate to the localhost where you can view your web page.
- Click on the predict button from bottom center, enter the inputs, click on the predict button, and see the result/prediction on the web.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\admin\OneDrive\Desktop\Rainfall_Prediction_Project> python app.py  
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit  
* Restarting with watchdog (windowsapi)  
* Debugger is active!  
* Debugger PIN: 373-571-501
```